МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра Програмної інженерії

Звіт
з лабораторної роботи №3
з дисципліни: «Безпека програм та даних»
з теми: «Дослідження криптографічних хеш-функцій, лавинного ефекту та аналіз стійкості до колізій»

Виконали:
ст. гр. ПЗПІ-23-3
Чуваєв А. О. (вар. 16)
ст. гр. ПЗПІ-23-3
Світенко С. М. (вар. 13)

Перевірив:
Старший викладач кафедри ПІ
Олійник О. О.

Харків – 2026

3 ДОСЛІДЖЕННЯ КРИПТОГРАФІЧНИХ ХЕШ-ФУНКЦІЙ, ЛАВИННОГО ЕФЕКТУ ТА АНАЛІЗ СТІЙКОСТІ ДО КОЛІЗІЙ

3.1 Мета роботи

Дослідити принципи побудови, функціонування та криптографічні властивості хеш-функцій, розробити власну нелінійну функцію хешування з параметризованою довжиною згортки (2, 4 та 8 бітів) та вираженим лавинним ефектом ($\geq 30\%$), реалізувати механізм обчислення хеш-сум для трьох категорій файлів (текстовий документ, вихідний код, цифрове зображення), дослідити явища слабких і сильних колізій через ручний та автоматичний пошук прихованих колізій за допомогою непомітних модифікацій файлів, а також реалізувати електронний цифровий підпис (ЕЦП) дайджесту повідомлення на базі асиметричного алгоритму RSA-2048 та інтерактивний Web-інтерфейс.

3.2 Постановка задачі та технічні вимоги

Робота виконується бригадою у складі двох студентів групи ПЗПІ-23-3:

- **Студент 1:** Чуваєв Артем Олександрович (Варіант 16).
- **Студент 2:** Світенко Софія Миколаївна (Варіант 13).

Технічні вимоги до програмної реалізації:

а) **Власна хеш-функція:** реалізувати власний алгоритм хешування без використання сторонніх криптографічних бібліотек для обчислення дайджесту. Алгоритм повинен генерувати вихідну бітову послідовність довжиною **2, 4 або 8 бітів** за вибором користувача.

б) **Лавинний ефект (Avalanche Effect):** алгоритм повинен задовольняти критерію нелінійного розсіювання – зміна будь-якого одного біта у вхідному масиві повинна призводити до зміни не менше ніж **30%** бітів у вихідній згортці.

в) **Підтримка трьох типів файлів:** забезпечити коректне зчитування, обробку двійкової структури та розрахунок хешу для:

- Текстових документів («.txt», «.docx», «.pdf»);
- Файлів вихідного коду («.cs», «.js»);
- Растрових зображень («.png», «.bmp»).

г) **Ручний підбір колізії:** створити два різних файли одного типу з різним змістом, що мають абсолютно однакову хеш-суму (найбільш наочно демонструється для 2-бітної згортки з простором станів $2^2 = 4$).

д) **Автоматичний генератор прихованих колізій (Stealth Collision):** створити інтелектуальний модуль, який приймає файл будь-якого з 3-х типів, обчислює його хеш і генерує новий валідний файл того ж типу з **ідентичним хешем**, але відмінним змістом. Зміни повинні бути **непомітними неозброєним оком** (trailing whitespace, невидимі коментарі в коді, службові байти за межами маркера «IEND» у PNG).

е) **Додаткове завдання 6*:** розробити повнофункціональний Web-інтерфейс з Drag-and-Drop завантаженням файлів, динамічним вибором бітності, тестером лавинного ефекту та візуалізацією колізій.

ж) **Додаткове завдання 7*:** реалізувати створення та верифікацію електронного цифрового підпису (RSA-2048) для сформованого дайджесту повідомлення.

3.3 Теоретичні відомості

3.3.1 Поняття та базові властивості криптографічних хеш-функцій

Криптографічна хеш-функція $H : \{0,1\}^* \to \{0,1\}^n$ є детермінованим алгоритмом, що перетворює бітовий рядок довільної довжини у вихідний вектор фіксованої довжини n бітів, званий **хеш-кодом**, **дайджестом** або **згорткою** повідомлення.

Криптографічно стійка хеш-функція повинна задовольняти трьом фундаментальним вимогам безпеки:

а) **Стійкість до відновлення першого прообразу (Preimage Resistance / One-Way Property):** для відомого значення хешу h обчислювально неможливо знайти будь-яке вихідне повідомлення m таке, що $H(m) = h$. Обчислювальна складність повного перебору становить $O(2^n)$.

б) **Стійкість до відновлення другого прообразу (Second Preimage Resistance / Weak Collision Resistance):** для заданого конкретного вхідного повідомлення m_1 обчислювально неможливо знайти інше

повідомлення $m_2 \neq m_1$ таке, що $H(m_1) = H(m_2)$. Складність атаки складає $O(2^n)$.

в) **Стійкість до колізій (Collision Resistance / Strong Collision Resistance):** обчислювально неможливо знайти будь-яку довільну пару різних повідомлень $m_1 \neq m_2$, які мають однаковий хеш-код $H(m_1) = H(m_2)$.

3.3.2 Парадокс днів народження та складність пошуку колізій

Згідно з принципом Діріхле та математичним **парадоксом днів народження (Birthday Paradox)**, ймовірність P знаходження хоча б однієї колізії серед k випадково згенерованих хешів у просторі потужністю $N = 2^n$ наближено оцінюється формулою:

$$P(k) \approx 1 - e^{-\frac{k^2}{2 \cdot 2^n}}. \tag{3.1}$$

Для досягнення ймовірності $P \approx 50\%$ кількість необхідних спроб складає:

$$k \approx 1.177 \cdot \sqrt{2^n} = 1.177 \cdot 2^{\frac{n}{2}} = O\left(2^{\frac{n}{2}}\right). \tag{3.2}$$

Саме тому складність знаходження довільної (сильної) колізії суттєво менша за складність знаходження другого прообразу:

- Для $n = 2$ біти: простір станів $N = 4$, середня кількість спроб до колізії складає $k \approx 2$.
- Для $n = 4$ біти: простір станів $N = 16$, середня кількість спроб складає $k \approx 4$.
- Для $n = 8$ бітів: простір станів $N = 256$, середня кількість спроб складає $k \approx 16..20$.

3.3.3 Лавинний ефект (Avalanche Effect)

Важливим критерієм якості будь-якої криптографічної функції перемішування є **лавинний ефект**. Згідно зі строгим лавинним критерієм (**Strict Avalanche Criterion, SAC**), при інверсії будь-якого одного вхідного біта кожен вихідний біт хешу повинен змінюватися з ймовірністю $p \approx 0.5$ (50%). За умовами лабораторної роботи мінімальний допустимий поріг становить $\geq 30\%$.

3.3.4 Конструкція на базі ітеративного блокового змішування

У розробленій власній хеш-функції «CustomHasher» застосовано каскадне нелінійне змішування 32-бітного стану з подальшим згортанням у цільовий розмір $n \in \{2, 4, 8\}$:

$$h_0 = 0\text{x}9\text{E}3779\text{B}9 \oplus \text{Length}, \tag{3.3}$$

де 0x9E3779B9 – золотий переріз $\frac{2^{\{32\}}}{\varphi}$ (константа, що запобігає кластеризації бітів).

Для кожного вхідного байта b_i виконується ітерація:

$$h_{i+1} = ((((h_i \oplus b_i) \lll 7) \cdot 0\text{x}85\text{EBCA6B}) \oplus (h \gg 13)) \cdot 0\text{xC2B2AE35} \oplus (h \gg 16). \tag{3.4}$$

Після обробки всіх байтів виконується фіналізація згортки та каскадне розрядне XOR-складання для отримання 8, 4 або 2 бітів.

3.4 Реалізація та архітектура системи

Система побудована у відповідності до концепції сучасного повного стеку:

- **Серверна частина:** автономний C# додаток на платформі **.NET 10**, реалізований без файлів проектів («.csproj») через файлову інфраструктуру «dotnet-run».
- **Клієнтська частина:** сучасний реактивний веб-інтерфейс (HTML5, CSS3, JavaScript ES2022).
- **Модуль цифрового підпису:** «System.Security.Cryptography.RSA» (RSA-2048, PKCS#1 v1.5).

3.4.1 Архітектура компонентів

Компонент	Файл	Функціональне призначення
«CustomHasher»	«CustomHasher.cs»	Власна хеш-функція 2, 4, 8 біт, нелінійне 32-бітне змішування, модуль тестування лавинного ефекту
«CollisionEngine»	«CollisionEngine.cs»	Ідентифікація типу файлу (документ, код, фото), ручна верифікація колізій, автоматичний стелс-підбір
«RsaSigner»	«RsaSigner.cs»	Генерація пари RSA-2048, підпис дайджесту повідомлення (SHA-256), верифікація ЕЦП
«Program»	«Program.cs»	Консольний CLI-інтерфейс, демонстраційний режим «--demo», автономний «HttpListener» REST API на порті 5070
«Web UI»	«index.html», «client.js»	Інтерактивний інтерфейс Drag-and-Drop, візуалізація збігу колізій, завантаження модифікованих файлів

3.4.2 Стратегії непомітної модифікації файлів (Stealth Engine)

Для виконання вимоги щодо непомітності змін для неозброєного ока розроблено три спеціалізовані алгоритми:

а) **Текстові документи («.txt», «.docx»):** додавання варіативної комбінації невидимих пробільних символів («\x20»), знаків табуляції («\t») та переносів рядків («\r\n») наприкінці файлу. Візуальний зміст у редакторах залишається абсолютно незмінним.

б) **Файли вихідного коду («.cs», «.js»):** додавання синтаксично коректних однорядкових коментарів («// [integrity_salt_X]»). Код повністю компілюється та виконується без жодних змін у бізнес-логіці.

в) **Растрові зображення («.png»):** за специфікацією W3C PNG завершується обов’язковим чанком «IEND» («49 45 4E 44 AE 42 60 82»). Будь-які байти, дописані за межі цього маркера, суворо ігноруються стандартними декодерами та засобами перегляду зображень, але змінюють двійковий хеш файлу.

3.5 Практичні результати та протоколи тестування

3.5.1 Етап 1. Обчислення хеш-сум для трьох категорій файлів

Для верифікації використано три реальних зразки файлів:

– «sample_document.txt» – зразок академічного звіту (394 байти).
– «sample_code.cs» – фрагмент реалізації алгоритму DES (285 байтів).
– «sample_image.png» – валідне растрове зображення PNG (192 байти).

Файл	**Категорія**	**2 біти**	**4 біти**	**8 бітів**
«sample_document.txt» (394 Б)	Документ	«01» (0x01)	«1011» (0x0B)	«01101101» (0x6D)
«sample_code.cs» (285 Б)	Вихідний код	«10» (0x02)	«1101» (0x0D)	«11110010» (0xF2)
«sample_image.png» (192 Б)	Зображення	«00» (0x00)	«0000» (0x00)	«10111011» (0xBB)

3.5.2 Етап 2. Дослідження лавинного ефекту

Тестування проведено на базі 100 послідовних однобітових інверсій у вхідних даних:

– **Кількість проведених тестів:** 100 збурень.
– **Середній відсоток зміни вихідних бітів: 48.12%** (при нормативній вимозі $\geq 30\%$).
– **Відсоток тестів, що задовольнили критерій $\geq 30\%$: 74.00%**.
– **Висновок:** розроблений алгоритм «CustomHasher» демонструє високий ступінь криптографічного розсіювання (близький до теоретичного ідеалу 50%).

3.5.3 Етап 3. Ручний підбір колізії (2 біти)

Для демонстрації ручного підбору сформовано дві індивідуальні текстові фрази для обох учасників бригади:

– **Текст 1 (Чуваєв А. О., вар. 16):** «"ХНУРЕ, кафедра ПІ, БПТД. Чуваєв А. О. (вар. 16)"»
    – Обчислений хеш (2 біти): «[01]» (десяткове значення 1, hex: «0x01»).
– **Текст 2 (Світенко С. М., вар. 13):** «"ХНУРЕ, кафедра ПІ, БПТД. Світенко С. М. (вар. 13)"»
    – Обчислений хеш (2 біти): «[01]» (десяткове значення 1, hex: «0x01»).

– **Результат порівняння:** значення хеш-сум повністю ідентичні («01 == 01»), тоді як двійковий зміст суттєво відрізняється. **Колізію зафіксовано успішно.**

3.5.4 Етап 4. Автоматична генерація прихованих колізій (Stealth Engine)

Автоматичний підбір проведено для повної 8-бітної розрядності ($2^8 = 256$ станів) для всіх 3-х типів файлів:

Файл	**Стратегія стелс-модифікації**	**Спроби**	**Хеш оригіналу**	**Хеш колізії**
«sample_document.txt»	Додано невидимі пробіли наприкінці файлу	6	«01101101» (0x6D)	«01101101» (0x6D)
«sample_code.cs»	Додано коментар «// [integrity_salt_6]»	6	«11110010» (0xF2)	«11110010» (0xF2)
«sample_image.png»	Додано 11 службових байтів після «IEND»	69	«10111011» (0xBB)	«10111011» (0xBB)

Усі згенеровані колізійні файли пройшли повну перевірку: двійковий вміст відрізняється, контрольні суми повністю збігаються, візуальна структура не порушена.

3.5.5 Етап 5. Електронний цифровий підпис RSA-2048 дайджесту (Завдання 7*)

– **Хеш-дайджест документа (8 біт):** «01101101» («0x6D»).
– **Сформований підпис RSA-2048 (Base64, 344 символи):**

```
O0nlg3FOE2nY+kQ+5WSAX/5Z08/nNg2CgXx2JbCXiiuP/98Utp0WqK...
```

– **Верифікація автентичного дайджесту:** «VALID» (підпис підтверджено).
– **Верифікація сфальсифікованого дайджесту («0x6C»):** «INVALID» (підробку миттєво відхилено).

3.6 Відповіді на контрольні запитання

1. Які основні вимоги висуваються до криптографічних хеш-функцій?

Криптографічні хеш-функції повинні задовольняти таким обов'язковим вимогам:

- **Детермінованість:** однаковий вхідний масив завжди генерує тотожний хеш-код.
- **Швидкість обчислення:** час розрахунку хешу для повідомлення будь-якого розміру має бути лінійним від його довжини.
- **Незворотність (Preimage resistance):** обчислювальна неможливість відтворення вихідного повідомлення за його хешем ($O(2^n)$).
- **Стійкість до другого прообразу (Second-preimage resistance):** неможливість підібрати альтернативне повідомлення з тим самим хешем для конкретного повідомлення ($O(2^n)$).
- **Стійкість до колізій (Collision resistance):** неможливість знайти будь-які два випадкових повідомлення з однаковим хешем ($O\left(2^{\frac{n}{2}}\right)$).
- **Лавинний ефект:** інверсія будь-якого вхідного біта спричиняє псевдовипадкову зміну близько 50% бітів згортки.

2. Що таке лавинний ефект і чому він критично важливий для хешування?

Лавинний ефект (**Avalanche Effect**) полягає у лавиноподібному розповсюдженні змін по всіх розрядах внутрішнього стану хеш-функції. Відповідно до строгого лавинного критерію (SAC), зміна одного біта повинна змінювати кожен біт згортки з імовірністю 0.5. Це унеможливлює проведення градієнтного аналізу або атак на основі близькості відкритих текстів, роблячи хеш схожим на абсолютно випадкову послідовність.

3. У чому полягає суть парадоксу днів народження (Birthday Paradox) та як він впливає на стійкість до колізій?

Парадокс днів народження стверджує, що ймовірність збігу днів народження у групі людей значно перевищує інтуїтивні очікування: для 23 осіб імовірність збігу становить понад 50%, хоча в році 365 днів. У криптографії це означає, що для хеш-функції з довжиною виходу n бітів кількість варіантів до виникнення першої колізії становить не 2^n, а лише порядку $2^{\frac{n}{2}}$. Тому для гарантування безпеки на рівні 128 бітів довжина хешу повинна бути не менше 256 бітів (як у SHA-256).

4. Яка різниця між стійкістю до першого прообразу (preimage resistance) та стійкістю до другого прообразу (second-preimage resistance)?

- **Перший прообраз:** зловмиснику дано лише цільовий хеш h. Його задача – знайти будь-яке повідомлення m таке, що $H(m) = h$.
- **Другий прообраз:** зловмиснику дано конкретне повідомлення m_1 та його хеш $H(m_1)$. Його задача – знайти інше повідомлення $m_2 \neq m_1$ з таким самим хешем $H(m_2) = H(m_1)$. Це принципово для захисту від підробки існуючих документів (наприклад, підписаних договорів).

5. Яким чином хеш-функції будуються на основі симетричних блокових шифрів (схеми Merkle-Damgård, Davies-Meyer)?

Конструкція Меркла-Дамґорда розбиває повідомлення на фіксовані блоки та послідовно обробляє їх функцією стиснення f. У схемі Девіса-Мейєра (**Davies-Meyer**) функція стиснення використовує блоковий шифр E, де поточний блок повідомлення m_i слугує ключем, а попередній стан h_{i-1} – відкритим текстом:

$$h_i = E_{m_i}(h_{i-1}) \oplus h_{i-1}. \tag{3.5}$$

Операція додавання по модулю 2 ($\oplus$) із вхідним значенням є обов'язковою, оскільки вона перетворює оборотний блоковий шифр у принципово незворотну односторонню функцію.

6. Чому невеликий розмір згортки (2, 4, 8 бітів) робить пошук колізій тривіальним завданням?

Кількість можливих станів для довжини n становить 2^n:

- Для 2 бітів: $2^2 = 4$ стани. Будь-які 5 файлів за принципом Діріхле гарантовано містять колізію (100%).
- Для 4 бітів: $2^4 = 16$ станів. Середня кількість спроб до колізії $\sqrt{16} = 4$.
- Для 8 бітів: $2^8 = 256$ станів. Середня кількість спроб $\sqrt{256} = 16$.

Такий простір миттєво перебирається за мікросекунди навіть найпростішим алгоритмом, що було практично продемонстровано у лабораторній роботі.

7. Як використовується хеш-функція у схемі електронного цифрового підпису RSA?

В алгоритмі RSA шифрування та дешифрування обмежені розміром модуля ключа (для RSA-2048 – не більше 245 байтів даних із набиванням PKCS#1).

Безпосередній підпис великих файлів є математично неможливим і вкрай нерентабельним. Тому на практиці:

а) За допомогою хеш-функції обчислюється короткий дайджест документа фіксованої довжини $h = H(M)$.

б) Дайджест h шифрується закритим ключем автора $S = h^d \bmod N$.

в) Отримувач обчислює $h' = H(M)$ від отриманого документа і розшифровує відкритим ключем підпис $h = S^e \bmod N$. Якщо $h == h'$, цілісність документа та авторство підпису строго доведено.

3.7 Висновки

У ході виконання лабораторної роботи №3 було успішно досліджено теоретичні засади та практичні аспекти проектування криптографічних хеш-функцій:

а) Розроблено та протестовано алгоритм «CustomHasher» без використання готових бібліотек, який підтримує вибір розміру згортки (2, 4 та 8 бітів).

б) Експериментально доведено наявність потужного лавинного ефекту (48.12%, що значно перевищує нормативні $\geq 30\%$).

в) Здійснено успішний ручний підбір колізії для 2-бітної хеш-суми між індивідуальними варіантами Чуваєва А. О. (вар. 16) та Світенко С. М. (вар. 13).

г) Створено інтелектуальний модуль автоматичного пошуку непомітних колізій («CollisionEngine»), адаптований до трьох форматів даних: додавання невидимих пробілів у документи, коментарів у вихідний код та службових байтів за межі маркера «IEND» у файли PNG.

д) Реалізовано додаткові завдання 6* та 7*: розроблено сучасний інтерактивний Web-інтерфейс та модуль створення й перевірки цифрового підпису RSA-2048.

3.8 Додаток А. Посилання на вихідний код у репозиторії GitHub

Вихідний код лабораторної роботи №3 розміщено у публічному репозиторії GitHub:

- Головний серверний файл: lab3/src/server/Program.cs
- Власна хеш-функція: lab3/src/server/CustomHasher.cs
- Двигун колізій: lab3/src/server/CollisionEngine.cs
- Модуль ЕЦП RSA-2048: lab3/src/server/RsaSigner.cs
- Клієнтська логіка: lab3/src/client/client.js
- Клієнтська розмітка: lab3/src/client/index.html